# AI-ConneX System Flow and Technology Architecture

## 1. Purpose

This document defines the end-to-end AI-ConneX system flow and identifies the technology used at each stage.

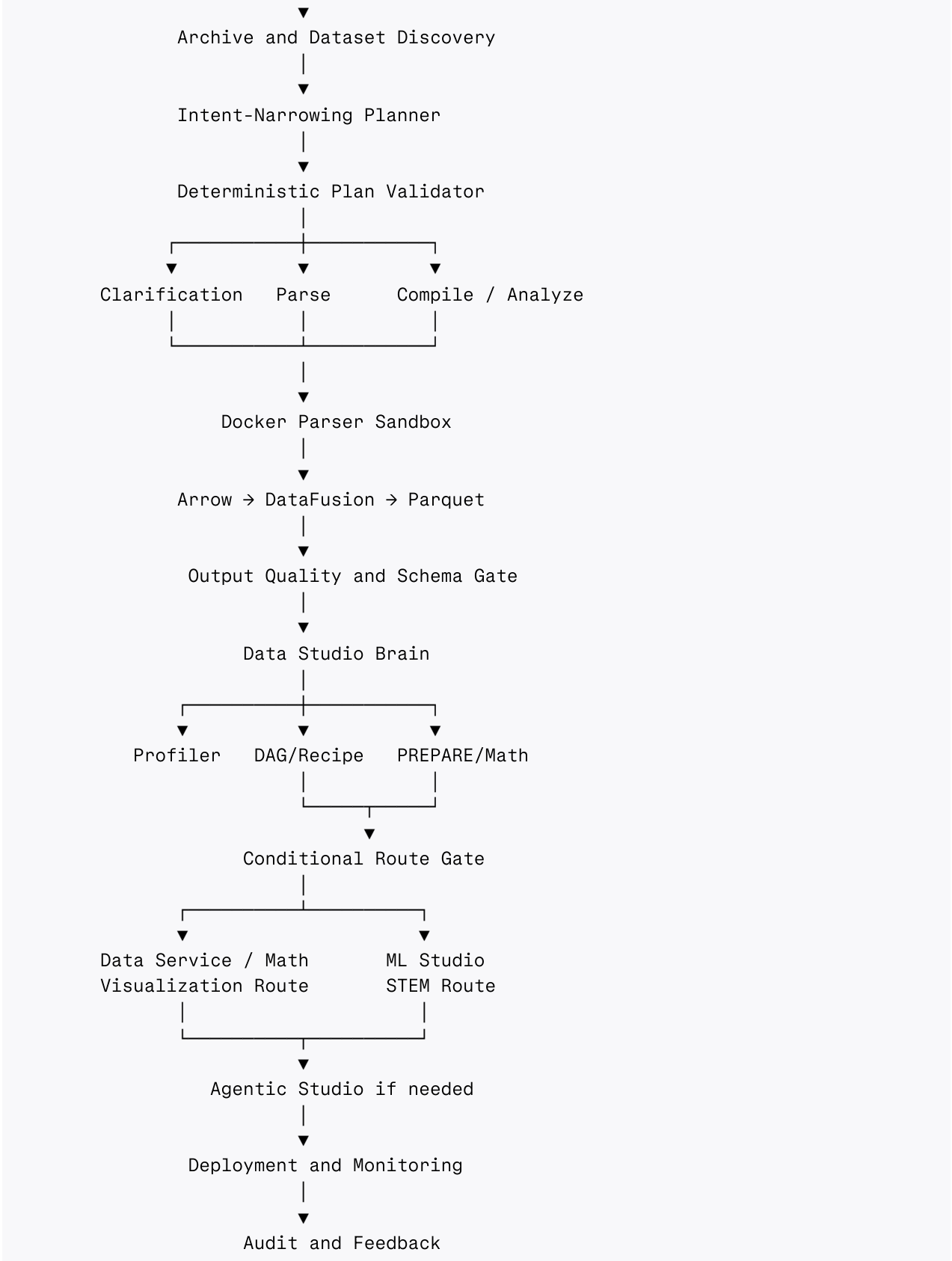
The architecture supports:

- Natural-language interaction through Jane.
- Intent classification and clarification.
- File, archive, API and telemetry ingestion.
- Conditional parsing versus full compilation.
- Docker-isolated dataset processing.
- Apache Arrow, DataFusion and Parquet data processing.
- Data Studio Brain analysis.
- Non-ML visualization and analysis.
- ML Studio model workflows.
- Agentic Studio agent creation and execution.
- Deployment, monitoring, audit and feedback.

**Global rule:** Agents propose typed plans. Deterministic services, policy checks and validators decide what may execute.

## 2. Complete System Flow

```
USER / INDUSTRIAL SYSTEM
          |
          ├── Jane / NLP session
          ├── File upload
          ├── API connector
          ├── MQTT/OPC UA telemetry
          └── Existing dataset reference
                        |
                        ▼
              API Gateway and Intake Service
                        |
                        ▼
              Authentication + Tenant Context
                        |
                        ▼
              Intent / Asset / Dataset Registration
                        |
```


Archive and Dataset Discovery
Intent-Narrowing Planner
Deterministic Plan Validator
Clarification
Parse
Compile / Analyze
Docker Parser Sandbox
Arrow → DataFusion → Parquet
Output Quality and Schema Gate
Data Studio Brain
Profiler
DAG/Recipe
PREPARE/Math
Conditional Route Gate
Data Service / Math Visualization Route
ML Studio STEM Route
Agentic Studio if needed
Deployment and Monitoring
Audit and Feedback

## 3. Technology Overview

| Layer | Primary technology | Why it is used |
|---|---|---|
| User interface | React, Next.js, TypeScript | Industrial workspace, stage tabs and visualizations |
| Conversational API | FastAPI, Uvicorn | Python API for Jane and workflow services |
| LLM routing | Small local/managed LLM with constrained decoding | Intent classification and structured routing |
| Agent planning | LangGraph | Stateful planning, clarification, interrupts and routing |
| Code/SQL proposal | Qwen2.5-Coder or equivalent | Generates proposals only; never receives production authority |
| Narration | Phi-4-mini or equivalent | Converts validated results into readable explanations |
| Contracts | Pydantic + JSON Schema | Typed service, artifact and agent contracts |
| Session state | Redis + PostgreSQL | Temporary state plus durable workflow/workspace state |
| Raw artifacts | MinIO/S3-compatible storage | Immutable uploads and large artifacts |
| Archive inspection | Python `zipfile`/`tarfile` | Metadata inspection and controlled streaming reads |
| Columnar memory | Apache Arrow / PyArrow | Interoperable in-memory data representation |
| Query and compilation | Apache DataFusion | SQL/DataFrame processing over CSV and Parquet |
| Canonical storage | Apache Parquet | Typed, compressed, partitioned datasets |
| Data quality | Great Expectations | Versioned expectation suites and validation reports |
| Data processing | Polars | Fast DataFrame and lazy transformations |
| Workflow scheduling | Apache Airflow | Scheduled batch pipelines and retries |
| Sandbox | Docker | Disposable parser and transformation execution |
| Time-series storage | TimescaleDB/InfluxDB | Sensor history and telemetry queries |
| Event streaming | MQTT/Kafka | Industrial events, telemetry and replayable streams |
| Graph | Neo4j | Asset topology and industrial relationships |
| Vector retrieval | pgvector/Qdrant/OpenSearch | Scoped RAG retrieval |
| Feature registry | Internal registry/Feast where needed | Feature definitions and offline/online consistency |
| Experiment tracking | MLflow | Metrics, experiments, model registry and lineage |
| ML | scikit-learn, AutoGluon, AutoTS, PyCaret | Classical ML, AutoML and time-series workflows |
| Deep learning | PyTorch, TensorFlow, Hugging Face | Vision, NLP and deep-learning training |
| Model serving | ONNX Runtime/KServe/Triton | Portable and scalable inference |
| Deployment | Docker Compose initially; Kubernetes later | Local/simple deployment first, scale-out later |

| Layer | Primary technology | Why it is used |
|---|---|---|
| CI/CD | GitHub Actions, Docker, Trivy, Syft, Cosign | Testing, scanning, SBOM and signed artifacts |
| Observability | OpenTelemetry, Prometheus, Grafana | Traces, metrics, logs and dashboards |
| Secrets | Vault or cloud KMS | Externalized credentials and key management |
| Policy | OPA/Rego or deterministic policy service | Authorization and safety decisions |

## 4. Entry Layer: Jane and Direct Data Intake

### 4.1 Jane / NLP entry

```
User message
  → session resolver
  → conversation context
  → goal/entity/constraint extraction
  → confidence evaluation
  → structured Intent Envelope
```

**Technology:**

- React/Next.js for the chat UI.
- FastAPI for the conversation API.
- Redis for active session state.
- PostgreSQL for durable sessions and workspaces.
- Small LLM for classification.
- Pydantic/JSON Schema for the Intent Envelope.
- LangGraph for multi-turn clarification.

### 4.2 Direct data entry

```
Upload / connector / telemetry
  → API Gateway
  → identity and tenant resolution
  → SHA-256 hash
  → immutable source registration
  → quarantine storage
```

**Technology:**

- FastAPI multipart handling.
- Python archive libraries.
- MinIO/S3-compatible object storage.
- PostgreSQL metadata.

- OpenTelemetry request tracing.

### 5. Identity, Tenant and Policy Layer

Every request must carry a verified context:

```
tenant_uid
user_uid
site_scope
asset_scope
roles
environment
policy_version
correlation_uid
```

**Technology:**

- OIDC/OAuth2 identity provider.
- FastAPI authentication middleware.
- PostgreSQL Row-Level Security where appropriate.
- OPA/Rego or policy service.
- Vault/KMS for secrets.
- Tenant-prefixed object, cache, vector and graph access.

**Rule:** client-supplied tenant IDs are not trusted. The server derives tenant context from verified identity and membership.

### 6. Intent Classification and Narrowing

#### 6.1 Intent classifier

The classifier identifies the current request, not the entire session history.

```
Current user message
  → routing model
  → constrained structured output
  → confidence check
  → route candidate
```

Example intent labels:

```
DATA_INGESTION
DATA_PROFILE
DATA_QUALITY
DATA_VISUALIZATION
MATH_ANALYSIS
```

```
MODEL_BUILD
MODEL_INFERENCE
AGENT_CREATE
AGENT_EXECUTE
DEPLOYMENT
MONITORING
CLARIFICATION_REQUIRED
```

**Technology:**

- Qwen3-4B or another small routing model.
- GBNF/JSON grammar or equivalent constrained decoding.
- llama.cpp or compatible local inference runtime.
- Pydantic validation after decoding.
- Deterministic route policy after model output.

Constrained decoding guarantees valid output structure; it does not guarantee correct business meaning or authorization.

### 6.2 Intent-narrowing planner

```
DISCOVER
  → UNDERSTAND
  → NARROW
  → PROPOSE
  → VALIDATE
  → CLARIFY or FINALIZE
```

**Technology:**

- LangGraph.
- Qwen2.5-Coder for SQL/recipe proposals where required.
- PostgreSQL/Redis for resumable state.
- Pydantic plan contracts.
- Plan Validator for deterministic checks.

Possible plan results:

```
PARSE_ONLY
PROFILE_ONLY
COMPILE
MATH_ANALYSIS
PREPARE
COMPILE_THEN_PROFILE
ROUTE_TO_ML
ROUTE_TO_AGENTIC
NEEDS_CLARIFICATION
```

```
NEEDS_USER_CORRECTION
BLOCK
```

## 7. Dataset Discovery and Conditional Compilation

Compilation is conditional in this architecture.

### 7.1 Lightweight discovery

```
Archive metadata
  → member names and sizes
  → candidate formats
  → sample headers
  → candidate timestamps and identifiers
  → security findings
```

**Technology:**

- Python `zipfile` and `tarfile`.
- MIME/magic-byte detection.
- PyArrow schema inspection.
- DataFusion file inspection.
- Polars lazy scans.

The discovery step should not load the complete archive.

### 7.2 Parse-only path

```
Archive
  → safe streaming parser
  → schema validation
  → Arrow batches
  → Parquet
  → output validation
```

Use when files are already compatible and no join, merge or unit harmonization is required.

### 7.3 Full compilation path

```
Discovery
  → typed plan
  → DataFusion SQL / Arrow operations
  → append/join/merge
  → schema harmonization
  → type/time/unit normalization
  → quality validation
```

```
  → lineage
  → canonical Parquet
```

Use when multiple datasets need composition or semantic alignment.

### 7.4 User-correction path

```
Ambiguous or unsafe input
  → explain findings
  → ask for mapping or corrected upload
  → wait for user response
  → resume with new intent/data version
```

## 8. Docker Parsing Sandbox

Docker is the simpler sandbox for the current deployment stage.

```
Job Manager
  → disposable parser container
  → read-only input mount
  → controlled output mount
  → network disabled by default
  → CPU/RAM/disk limits
  → timeout
  → result manifest
  → container removal
```

**Technology inside the image:**

- Python archive parser.
- Polars or pandas chunked reading.
- Apache Arrow/PyArrow.
- Apache DataFusion.
- Apache Parquet.
- Pydantic output validation.
- Great Expectations validation.

**Container restrictions:**

- Non-root user.
- No privileged mode.
- No host network.
- No Docker socket.
- No arbitrary host mounts.

- Read-only input.
- Dedicated output directory.
- No runtime package installation.
- No arbitrary user code execution.
- No direct production storage writes.

Docker is an isolation layer, not a complete hostile-code boundary. Use prebuilt, signed parser images only.

## 9. Apache Data Plane

### 9.1 Apache Arrow / PyArrow

Use Arrow as the interchange format:

```
Parser
  → Arrow RecordBatch/Table
  → DataFusion
  → Arrow result
  → Parquet
```

Arrow avoids repeatedly converting between unrelated in-memory formats.

### 9.2 Apache DataFusion

Use DataFusion for:

- CSV and Parquet reads.
- SQL projections.
- Joins.
- Aggregations.
- Partition filters.
- Dataset inspection.
- Profiling scans.
- Conditional compilation.

Use parameterized query templates or validated query plans, not unrestricted agent-generated SQL.

### 9.3 Apache Parquet

Use Parquet for:

- Parsed datasets.
- Compiled datasets.
- Prepared datasets.
- Feature datasets.
- Mathematical-analysis outputs.

The output must include schema and lineage metadata alongside the Parquet dataset.

### 9.4 Apache Airflow

Use Airflow for scheduled and batch workflows:

```
scheduled archive reprocessing
nightly profiling
batch data-quality checks
historical backfills
model retraining
drift evaluations
usage/audit rollups
```

Do not use Airflow as the primary conversational routing engine.

### 9.5 Apache Beam, Kafka and Iceberg

Add these only when workload requirements justify them:

- Apache Kafka: multiple live sites, durable event replay and decoupled telemetry ingestion.
- Apache Beam: portable distributed batch/stream processing.
- Apache Iceberg: snapshots, schema evolution, time travel and concurrent table management.

## 10. Data Studio Brain

```
Compiler / Parser
        ↓
Data Profiler
        ↓
DAG / Relationship Analysis when needed
        ↓
Recipe Orchestrator
        ↓
Selected node execution
```

### 10.1 Profiler

**Technology:**

- DataFusion.
- Arrow/PyArrow.
- Polars.
- NumPy/SciPy.
- Great Expectations for validation inputs.
- PostgreSQL for profile metadata.

Outputs:

```
profile.json
quality.json
temporal.json
semantic.json
manifest-UID.json
report.json
```

### 10.2 DAG Assigner

**Technology:**

- PostgreSQL registry.
- DataFusion/Arrow profile access.
- NetworkX for graph validation.
- Pydantic/JSON Schema.
- Deterministic scoring service.
- Optional constrained LLM explanation.

The LLM may help explain or rank candidates, but it may only return DAG IDs present in the versioned registry.

### 10.3 Recipe Orchestrator

**Technology:**

- PostgreSQL recipe registry.
- Pydantic recipe contracts.
- NetworkX/DAG validation.
- Airflow for scheduled execution.
- Docker for isolated task execution.
- Arrow/Parquet artifacts.

The recipe registry remains read-only during execution.

### 11. PREPARE Node

PREPARE makes data valid and execution-ready. It does not train models.

```
Validate schema
  → validate expectations
  → handle missing values
  → handle duplicates
  → normalize timestamps
  → normalize units
  → apply domain rules
  → quarantine unresolved records
  → revalidate
  → write prepared Parquet
```

**Technology:**

- Polars for transformations.
- Arrow/Parquet for data movement and storage.
- DataFusion for large scans and joins.
- Great Expectations for declarative checks.
- Pandera only if a focused DataFrame schema check is needed.
- scikit-learn only for transformations that are specifically part of a training pipeline; do not use it as the general ingestion validator.
- Docker for isolation.

Every PREPARE run emits:

```
prepared_dataset.parquet
prepared_manifest.json
validation_results.json
prepare_lineage.json
quarantine.parquet
prepare_report.json
schema_fingerprint.json
```

Outliers should be flagged or quarantined by default, not blindly deleted.

### 12. Non-ML Data Studio Route

```
Profiler
  → optional relationship/DAG analysis
  → Recipe Orchestrator
  → PREPARE if needed
```

```
   → Mathematical/Physics Analysis
   → Data Service / Visualization
```

Do not invoke:

- STEM split.
- Model training.
- Model evaluation.
- Judge/Scorer for model selection.

### Mathematical/physics layer

**Technology:**

- NumPy.
- SciPy Signal.
- Polars.
- Arrow/Parquet.
- Registered formula/primitive registry.
- Plotly/ECharts/Vega-Lite for visualization.

Supported operations may include:

- FFT.
- PSD.
- RMS.
- Crest factor.
- Rolling statistics.
- Derivative/integral.
- Rate of change.
- Pressure-flow relationships.
- Energy calculations where units are valid.
- Threshold overlays.
- Cycle alignment.

Every output must include units, assumptions, sampling metadata, formula version and provenance.

### 13. ML Studio Route

```
Prepared/feature package
  → ML Clean Validator
  → Model Plan confirmation
  → DAG/Family candidate space
  → S.T.E.M.
  → Judge
  → Scorer
  → Model Registry
  → Deployment
```

**Technology:**

- scikit-learn.
- AutoGluon.
- AutoTS.
- PyCaret.
- Optuna.
- tsfresh where approved for time-series features.
- PyTorch/TensorFlow for deep learning.
- MLflow for tracking and registry.
- ONNX Runtime for portable serving.
- Airflow for scheduled retraining.
- Docker for training task isolation.

The model is not promoted solely because it has the highest metric. It must satisfy metric, resource, explainability, leakage, lineage and deployment policies.

### 14. Agentic Studio Route

```
Approved automation intent
  → Ground-State resolution
  → Agent Builder
  → Agent SPEC
  → signature/hash
  → Agent Runtime
  → RAG/KG/Telemetry/Model context
  → Policy and approval
  → Tool Gateway
  → industrial result
```

**Technology:**

- LangGraph for bounded agent state.

- Pydantic/JSON Schema for Agent SPEC.
- PostgreSQL for registry and durable state.
- Neo4j for industrial topology.
- pgvector/Qdrant/OpenSearch for RAG.
- TimescaleDB/InfluxDB for telemetry.
- MLflow/ONNX Runtime for approved model references.
- OPA/Rego or policy service.
- Docker for tool workers.
- OpenTelemetry for execution traces.

The agent never receives direct access to industrial systems. Side effects pass through the Tool Gateway.

## 15. Frontend and Backend Event Architecture

### 15.1 Backend

Use:

- FastAPI for APIs.
- PostgreSQL for durable workflow/workspace state.
- Redis for short-lived state and locks.
- OpenTelemetry for traces.
- SSE for server-to-frontend streaming initially.
- WebSocket only where true bidirectional high-frequency communication is needed.

### 15.2 Event types

Keep chat, state and actions separate:

```
chat.token
chat.completed
intent.updated
clarification.required
plan.proposed
plan.validated
stage.started
stage.progress
stage.ready
stage.warning
stage.failed
view.navigate
view.unlock
approval.required
approval.completed
```

```
artifact.created
execution.completed
```

Frontend tabs unlock only after an authorized backend event and artifact validation.

### 15.3 Two-axis state

```
Axis A: persistent workspace capability state
  data_profile_available
  prepared_dataset_available
  feature_package_available
  model_plan_available
  agent_spec_available
  deployment_allowed

Axis B: current per-message intent
  DATA_PROFILE
  DATA_VISUALIZATION
  MODEL_BUILD
  AGENT_CREATE
  DEPLOYMENT_STATUS
  CLARIFICATION
```

Axis A lives in PostgreSQL. Axis B is classified fresh on every message.

## 16. Agent and Code-Generation Policy

### 16.1 Model roles

| Model | Role | Restriction |
|---|---|---|
| Small routing model | Intent classification | Constrained output, no execution |
| Qwen2.5-Coder | SQL/recipe proposal | Sandbox only, no direct production writes |
| LangGraph planner | Dynamic workflow reasoning | Typed proposal only |
| Phi-4-mini | Narration | Receives validated results only |

### 16.2 Code/SQL proposal loop

```
Generate proposal
  → syntax validation
  → schema validation
  → policy validation
  → Docker dry-run
  → output validation
  → bounded repair if necessary
  → approve or fail
```

No arbitrary generated Python is promoted directly into the production recipe registry. Prefer registry recipes and allow-listed primitives.

## 17. Deployment Architecture

### 17.1 Current simpler deployment

```
Docker Compose
  ├── API service
  ├── Planner service
  ├── PostgreSQL
  ├── Redis
  ├── MinIO
  ├── DataFusion/parser worker
  ├── Profiling worker
  └── Observability collector
```

Docker containers should be independently restartable and use dedicated volumes.

### 17.2 Scheduled operations

Add Apache Airflow when scheduled workloads exist:

```
Airflow scheduler
  → DockerOperator
  → parser/profile/reprocess containers
  → output validation
  → manifest update
```

### 17.3 Future scale-out

Add Kubernetes, KubernetesPodOperator, Kafka, Beam and Iceberg only when:

- Multiple hosts are required.
- Tenant workloads need independent scaling.
- Live telemetry volume exceeds local workers.
- High availability requires orchestration.
- Concurrent writers require table-level transaction semantics.

## 18. CI/CD Architecture

### CI on every change

```
Source change
  → Ruff/formatting
  → mypy/Pyright
  → pytest
  → contract tests
  → parser security tests
  → archive replay tests
  → leakage tests
  → tenant-isolation tests
  → Great Expectations fixture validation
  → container build
  → Trivy scan
  → Syft SBOM
  → Cosign signature
```

### CD

```
Signed artifact
  → development
  → known-good ingestion tests
  → malicious archive tests
  → ambiguous intent tests
  → staging replay
  → human production approval
  → production deployment
```

### Test categories

- Intent-to-route tests.
- Parse-only route tests.
- Conditional compilation tests.
- Schema drift tests.
- ZIP Slip tests.
- ZIP bomb tests.
- Corrupt-file quarantine tests.
- Memory/timeout tests.
- Output promotion tests.
- Numeric narration verification.
- Cross-tenant access tests.
- Agent overreach tests.
- Airflow retry/replay tests.
- Model leakage tests.

### 19. Observability and Audit

Every execution should record:

```
execution_uid
correlation_uid
tenant_uid
user_uid
intent_uid
manifest_uid
recipe_id/version
parser_image_digest
input_hashes
output_hashes
status
warnings
errors
resource_usage
start/end timestamps
```

Use:

- OpenTelemetry for distributed tracing.
- Prometheus for metrics.
- Grafana for dashboards.
- PostgreSQL/object storage for audit records.
- MLflow for experiment/model records.
- Great Expectations for validation evidence.

The Monitor Page should show data health, pipeline health, model health, agent health and deployment health as separate signals.

### 20. Final Recommended Stack

### Initial production deployment

```
Frontend:
  React + Next.js + TypeScript

API:
  FastAPI + Uvicorn

Contracts:
  Pydantic + JSON Schema

Agent planning:
  LangGraph
```

```
Routing model:
  Qwen3-4B or approved small model with constrained decoding

Code/SQL proposal:
  Qwen2.5-Coder in Docker sandbox

Narration:
  Phi-4-mini using validated result objects

State:
  PostgreSQL + Redis

Storage:
  MinIO/S3-compatible object storage

Parsing:
  Python archive libraries + Polars

Columnar:
  Apache Arrow/PyArrow

Query:
  Apache DataFusion

Storage format:
  Apache Parquet

Quality:
  Great Expectations

Orchestration:
  Apache Airflow for scheduled jobs
  LangGraph/Job Manager for dynamic per-request jobs

ML:
  scikit-learn, AutoGluon, AutoTS, PyCaret, PyTorch as required

Feature extraction:
  Feature Registry; tsfresh only after validation and feature-budget controls

Tracking:
  MLflow

Serving:
  ONNX Runtime initially; KServe/Triton later if needed

Sandbox:
  Docker

Observability:
  OpenTelemetry + Prometheus + Grafana

Security:
  Vault/KMS + Trivy + Syft + Cosign
```

### Later Apache scale-out additions

```
Kafka
Apache Beam
Apache Iceberg
Apache Spark
Kubernetes
KubernetesPodOperator
```

## 21. Final System Rule

```
Jane understands intent.
The routing model classifies the current request.
LangGraph narrows and plans.
Pydantic validates contracts.
The Plan Validator decides what is executable.
Docker isolates parser execution.
Arrow represents data.
DataFusion queries and transforms.
Parquet stores canonical artifacts.
Great Expectations validates quality.
The Brain profiles and routes.
Airflow schedules recurring workflows.
ML Studio builds models.
Agentic Studio governs actions.
OpenTelemetry observes everything.
Human approval controls production promotion.
```

The architecture deliberately combines Apache infrastructure, established Python libraries and LLM components, but keeps the authority boundaries explicit:

> **LLMs propose. Apache and Python workers execute approved operations. Validators decide promotion. Policies authorize access. Humans approve consequential changes.**